

C19 — Platforms and Ecosystems

1. What interviewers test

"How would you grow a marketplace from nothing?" "You run a payments API; developers say integration is painful. What do you do?" "Design the review policy for an app store." "Your biggest developer is building a competing product. What now?" Platform questions combine strategy (C18), product craft and some technical fluency, and they are a favorite at companies that run platforms *Inferred*, because a platform PM's decisions affect people who are not on the team: developers, sellers, buyers and regulators. The interviewer wants to see whether you can name the participants and what each needs, explain why the platform becomes more valuable (or fails to start), set rules that balance speed, safety and trust, and see the risks that come from depending on, and competing with, your own ecosystem.

Where it shows up. *Inferred* from the round structures candidates commonly report: platform, marketplace and developer-product questions at Google, Meta, Microsoft, Amazon, Apple, Stripe and others, frequently in strategy or product-sense rounds, and for technical-leaning roles in API-product depth *[unverified: candidate-reported]*. On the job it is the work of PMs on marketplaces, app stores, payment and cloud platforms, and internal platforms.

What is scored. Whether you reason about more than one side at a time *Inferred*. Interviewers listen for a clear map of participants and the value each gets; an understanding of network effects with their limits (multi-homing, local networks); a credible plan for the cold start; API and developer-experience thinking; governance that names the tradeoffs and the economics (take rates); and a scan of ecosystem risks.

Six observable behaviors:

1. You map the platform's sides, what each gives and gets, and what links them.
2. You distinguish a two-sided market from a developer platform, and say which problems each brings.
3. You treat network effects as a mechanism with limits, and plan the cold start with a measure of liquidity.
4. You treat an API as a product and measure developer experience, for example the time to a first successful call.
5. You set governance (rules, review, take rate, openness) as tradeoffs with consequences for each side.
6. You scan for ecosystem risk: dependency, disintermediation, conflict with participants, and regulatory attention.

The failure this prevents. Six patterns end platform answers: naming "network effects" as a label, designing for one side only, a launch plan that spreads supply thin, an API treated as plumbing, rules set without thinking about developer trust, and ignoring what happens when the platform competes with its participants.

2. The concept from first principles

C19.1 What a platform is: two-sided markets versus developer platforms

A **platform** is a product that others build on or transact through, creating value for more than one group of participants. Two broad kinds are common, and many are both. FW-C19-01 is the map and FW-C19-02 chooses the type.

A **two-sided market** is a market with two distinct groups of participants who need each other, where the platform's value to each group depends on the size and quality of the other. A ride-hailing service (riders and drivers), a marketplace (buyers and sellers) and a job board (employers and candidates) are examples. The platform's job is matching and trust: finding the right counterpart, handling payment, and resolving disputes.

A **developer platform** is one that others build products on, through interfaces such as APIs (C21), software kits, and tools. An operating system, a cloud service and a payments API are examples. The platform's job is to be a reliable, well-documented foundation, and its participants build things the platform could not, which makes the whole more valuable.

Many platforms combine both: an app store has developers who build apps (a developer platform) and users who buy them (a market). Name the sides explicitly and ask, for each: what do they give the platform and each other, what do they get, what can they do instead, and what do they fear?

When not to use the platform lens. A product with many users but no interaction among them, and no outside builders, is not a platform. Calling it one imports the wrong problems (and hides the real ones).

Counterexample. Asked to grow a marketplace for local services, a candidate says: "It has network effects, so growth will compound. I'd launch in all major cities and spend on marketing." The interviewer asks what happens in a city with ten sellers and a thousand buyers. The candidate had not considered that a request in such a city rarely gets a good response, so buyers leave and do not return. The network effect is local and conditional, and a launch that spreads supply thinly starts nothing (C19.2).

Hidden risks. Two. *One-sided design:* optimizing for buyers (low prices, easy cancellation) can drive sellers out, and the reverse. *Platform thinking where there is no platform:* labeling an ordinary product a platform to sound strategic.

C19.2 Network effects, flywheels and the cold start

C18.5 defined a network effect. For platforms there are two kinds. A *same-side* effect: more users of one kind make the platform better for the same kind (a messaging app: more friends). A *cross-side* effect: more of one side makes the platform better for the other (more drivers shorten rider waits, and more riders raise driver earnings). A **flywheel** is a reinforcing loop in which growth on one side feeds growth on the other; the PM's job is to find where it starts and what slows it.

Limits. Network effects are often local (riders in one city do not benefit from drivers in another), can saturate (the thousandth seller adds little), and can be weakened by **multi-homing**, which is when participants use several platforms at once. If drivers can run two ride apps and buyers can check two marketplaces, no platform locks in the market, and competition turns on price, service and incentives. Check, for any claimed effect, how local it is, where it saturates, and whether participants multi-home.

The cold start. The **cold-start problem** is the difficulty a platform faces when its value depends on participants who will not join until the platform is already valuable. Tactics (FW-C19-03):

1. *Make it useful to one side alone* ("single-player mode"): a tool people use without the other side, so they arrive before the network does.
2. *Narrow the market* to one city, one category or one community, so that density is reachable.
3. *Seed the harder side*: pay, recruit or build supply yourself, and subsidize if needed.
4. *Borrow an existing network* (invite from a base, integrate with one) rather than starting empty.
5. *Measure liquidity*, not sign-ups.

Liquidity and density. Suppose each seller can fulfill a given request with probability 4% (invented). The chance that a buyer's request is matched with at least one of N sellers is $1 - 0.96^N$: 56% with 20 sellers, 71% with 30, 91% with 60, 96% with 80. Spreading 160 sellers over 8 cities (20 each) gives 56% matching everywhere; putting them in 2 cities (80 each) gives 96%. Density per market, not the total, decides whether buyers get what they came for and return. The practical rule: launch where you can reach the density threshold, prove it, then expand.

Why it works. The arithmetic of matching shows why a narrow launch beats a broad one.

When not to apply it. For pure developer platforms, where the cold start is about developer adoption (C19.3), not matching.

C19.3 APIs as product; developer experience

For a developer platform, the API (the defined way programs talk to the platform, C21) is the product, and developers are the users. **Developer experience** is the sum of what it takes for a developer to discover, understand, integrate and succeed with a platform: documentation,

examples, software kits, error messages, tooling, support and pricing. A platform with a powerful API and poor developer experience loses developers to one that is easier to start with.

Measure the first mile. The key measure is the *time to first successful call*: from signing up to the first working request. Break it into a funnel (FW-C19-04): discover, sign up, get credentials, make a first call, ship to production. Suppose 10,000 developers sign up (invented); 7,600 get credentials (76%), 5,320 make a first successful call (70% of those), and 3,000 reach production (56% of those): 30% overall. If better documentation and clearer errors raise the three steps to 82%, 85% and 72%, the final count is $10,000 \times 0.82 \times 0.85 \times 0.72 = 5,018$, or 50%. Each step is a place to look for friction: unclear sign-up, confusing authentication, a first example that fails, an error that gives no hint.

Design for stability. Developers build on your interface, and changes can break their products. Treat compatibility as a feature: version the API, give long notice and support for old versions, and publish a deprecation policy. A platform that keeps old code working earns trust (the S3 case in C18 is an example), and one that breaks developers loses them.

Other levers. Sandboxes to try without risk; examples in several languages; clear rate limits and pricing; a status page; a community and support route; analytics for developers on their own usage. The PM owns the developer's journey, reads support tickets and forum posts, and sits with developers during first integrations.

Why it works. Treating the first integration as the product finds the friction that decides adoption. **When not to invest first.** In a platform with few developers, invest in understanding the first ten, not in tooling for thousands.

C19.4 Governance: rules, quality, take rates, openness tradeoffs

A platform sets the rules for its participants, and the rules are product decisions with consequences for every side. FW-C19-05 is the dial.

Rules and review. What may be sold or published, what quality bar applies, how problems are handled. A stricter review protects users (safety, privacy, quality) and slows developers and may deter them; a lighter one speeds releases and raises the risk of harm. The tradeoff is among *speed* (time to approval), *safety* (harm prevented) and *developer trust* (predictability, fairness, a route to appeal). Design for all three: clear written rules, consistent decisions, fast turnaround on routine submissions, human review for hard cases, explanations for rejections, and an appeals path.

The take rate. The **take rate** is the share of transaction value a platform keeps. Too high and participants leave, raise prices or bypass the platform; too low and the platform cannot fund itself, its quality and its safety. Take rates vary widely by platform and category [V: Apple and Google app-store fee tiers]. Evaluate with a simple model: if raising the rate lowers the value of transactions through the platform (fewer listings, higher prices, more bypass), revenue may fall. Take an example (invented): at a 30% take rate, transaction value is 100 and revenue 30; at 20%, more developers and lower prices lift transaction value to 108 and revenue to 21.6; at 15%, value 112 and revenue 16.8. Revenue falls as the rate falls, so a cut must be justified by ecosystem health, regulation or competition, not revenue alone.

The numbers are assumptions; the discipline is to state how transaction value responds and what each rate is for. Tiered rates (lower for small participants, or for those who use more platform services) are common.

Openness. How open should the platform be? Open (anyone can build, sideload, interoperate) maximizes innovation and participation and increases fragmentation, security risk and quality variance. Closed (curated, controlled) gives consistency and safety and invites criticism and regulatory attention. Many platforms choose a middle path with compatibility requirements, certification or tiers of access. The right point depends on the stakes of harm and on how much the platform's value rests on consistency.

Why it works. Naming the three-way tradeoff and the economics lets you defend a policy.

When not to tighten. Where harm is low and speed matters, a lighter review with after-the-fact enforcement may serve better.

C19.5 Ecosystem risk: dependency, disintermediation, regulatory attention

Platforms face a set of recurring risks (FW-C19-06):

- **Dependency, in both directions.** Participants depend on the platform (a policy change can end a business), and the platform depends on a few large participants (if its top developers leave, users follow). Concentration is a measure: what share of revenue or usage comes from the top 10?
- **Disintermediation.** Disintermediation is participants bypassing the platform to transact directly after being matched, which reduces the platform's revenue and its data. It is likely where the platform's value is mostly the introduction (a one-time match) and repeat transactions are easy to do off-platform. Defenses: provide value that continues (payments, protection, insurance, scheduling, reviews), and make staying cheaper than leaving.
- **Conflict with participants.** When a platform offers its own products alongside participants' (own-brand goods, first-party apps), participants worry about unfair treatment, especially use of their data and placement. Trust requires clear rules and, often, structural safeguards.
- **Regulatory attention.** Large platforms draw scrutiny on fees, openness, competition and data. Rules differ by country and change [as of 2026-10]. A PM should know where the platform is exposed and design with that in mind.
- **Security and quality incidents.** One bad actor can harm users and the whole platform's reputation. Review, monitoring and fast takedown matter.
- **Multi-homing and substitution.** If participants can use rivals easily, retention rests on service quality and incentives.

Regional Lens: India. Open networks and shared payment rails change platform competition in India. The Unified Payments Interface (C09) lets payments move across many apps, and government-backed open-network initiatives for commerce aim to let buyers and sellers on different apps transact with each other (ONDC is the best known; the trade ministry's DPIIT launched it in April 2022) [V: Reuters, 29 April 2022]. The consequence for a platform PM is that lock-in through payments or a closed network is weaker, and competition turns more on service quality, trust and price. Check the current state of any initiative before relying on it.

C19.6 Platform metrics

Metrics for a platform must cover each side and the interaction between them (FW-C19-07), with guardrails:

Area	Examples	Why
Supply	Active sellers or developers; new supply per week; supply quality	Without supply there is nothing to offer
Demand	Active buyers or users; new demand; repeat rate	Without demand supply leaves
Interaction (liquidity)	Match rate; time to match; fill rate; conversion from search to transaction	The platform's core value
Quality and trust	Ratings; disputes per 1,000 transactions; fraud rate; safety incidents	Trust sustains participation
Economics	Transaction value; take rate; contribution margin per transaction; payback	The business must work
Developer experience	Time to first call; share reaching production; API error rate; support tickets per developer	Developer adoption
Ecosystem health	Concentration of the top 10 participants; multi-homing; churn of supply; disintermediation proxies	Resilience

Choose a primary metric that reflects the platform's core value (often a measure of successful transactions), and watch the others as guardrails so that growth on one side does not hide trouble on another. A single metric such as gross transaction value can rise while quality, trust and developer satisfaction fall (C05, Goodhart).

C19.7 Case: a mobile app store as a platform

Section 10 contains a reconstructed case of an app store, built from public documents: the sides (developers and users), the rules, the review process, the revenue share and how it changed, and the risks. It is labeled **Reconstructed** because the analysis is an inference from public information and not from internal data.

C19.8 Answer architecture

Section 4 sets out the answer architecture for platform questions, with types for growing a marketplace, an API product, platform policy, a key participant turning competitor, and platform metrics.

C19.9 Worked examples

Section 5 gives five worked examples: an API's developer funnel, an openness decision, a take-rate analysis, a cold start by density and platform health metrics. Section 11 adds seven questions with model answers, including questions set on Apple, Microsoft and Stripe products.

3. Frameworks

Eight frameworks, set out below. The sources-of-advantage test (FW-C18-05), the strategy spine (FW-C18-02), the unit-economics sketch (FW-C18-06), the metric card (FW-C05-01), the rollout ladder (FW-C17-02) and the ethical nudge test (FW-C10-06) are reused, not repeated.

FW-C19-01 · The platform map · Original

- **Inputs:** a platform and its participants.
- **Outputs:** a map of sides: what each gives and gets, alternatives, fears, and the interface between them (rules, API, payments); the value loop; the first constraint.
- **Assumptions:** you can name the sides and their motives.
- **Tradeoffs:** a map simplifies; add sub-sides (large and small developers).
- **Failure modes:** one-sided design; a label with no mechanism; forgetting regulators and users.
- **Worked example:** the map in Worked examples 1 and 3 and CS-C19-02.

Side	Gives	Gets	Alternatives	Fears
------	-------	------	--------------	-------

FW-C19-02 · The platform type chooser · Original

- **Inputs:** a product that others build on or transact through.
- **Outputs:** a classification (two-sided market, developer platform, or both) with the problems each brings and the main metric.
- **Assumptions:** the participants are identifiable.
- **Tradeoffs:** hybrids carry both sets of problems.
- **Failure modes:** calling a product a platform without outside builders or counterparties.
- **Worked example:** IQ-C19-01.

Type	Core job	Hard problem	Primary metric
Two-sided market	Match and build trust	Cold start; liquidity; disintermediation	Successful transactions
Developer platform	Be a reliable, easy foundation	Developer experience; stability; governance	Active developers shipping to production
Hybrid (e.g., app store)	Both	Review policy; fees; conflict with participants	Value to both sides together

FW-C19-03 · The cold-start playbook · Original

- **Inputs:** a platform with no network.
- **Outputs:** a plan: single-player value, a narrow starting market, which side to seed and how, a borrowed network if any, and a liquidity target with the density needed.
- **Assumptions:** a match probability per supplier can be estimated.
- **Tradeoffs:** a narrow start limits the early market and creates density.
- **Failure modes:** spreading thin; counting sign-ups; no plan to seed the hard side.
- **Worked example:** Worked example 4 and CS-C19-01.

Formula: $P(\text{match}) = 1 - (1 - p)^N$, where p is the chance one supplier fulfills a request and N is the suppliers in the market.

FW-C19-04 · The developer-experience funnel · Original

- **Inputs:** a developer product.
- **Outputs:** a funnel (discover, sign up, credentials, first successful call, production), time to first call, and the friction at each step with fixes.
- **Assumptions:** the steps can be instrumented.
- **Tradeoffs:** optimizing the first mile helps new developers and may neglect experienced ones.
- **Failure modes:** measuring sign-ups; ignoring error messages; breaking changes without notice.
- **Worked example:** Worked example 1.

FW-C19-05 · The governance dial · Original

- **Inputs:** a platform rule or fee decision.
- **Outputs:** the options along speed, safety and developer trust (and openness and price), the effect on each side, a recommendation with measures and a review date.
- **Assumptions:** effects on each side can be reasoned about or tested.
- **Tradeoffs:** every setting helps one side and costs another.
- **Failure modes:** deciding for one side; no appeals path; fee changes without transition.
- **Worked example:** Worked examples 2 and 3 and IQ-C19-05.

Dial	Toward one end	Toward the other
Review	Strict: safety, slower	Light: speed, risk
Openness	Closed: consistency	Open: innovation, risk
Take rate	High: funding	Low: participation

FW-C19-06 · The ecosystem risk scan · Original

- **Inputs:** a platform and its participants.
- **Outputs:** a table of risks (dependency, concentration, disintermediation, conflict with participants, regulatory, security, multi-homing) with likelihood, impact, an indicator and a mitigation.
- **Assumptions:** indicators can be measured.
- **Tradeoffs:** mitigations cost revenue or control.
- **Failure modes:** a list without indicators; ignoring concentration.
- **Worked example:** the scan in CS-C19-02 and IQ-C19-06.

Risk	Indicator	Mitigation
------	-----------	------------

FW-C19-07 · The platform health metrics set · Original

- **Inputs:** a platform and its core value.
- **Outputs:** a primary metric tied to successful transactions or shipped integrations, metrics for supply, demand, interaction, quality, economics and ecosystem health, and guardrails.
- **Assumptions:** each side's activity is observable.
- **Tradeoffs:** many metrics invite confusion; keep one primary and a few guardrails.
- **Failure modes:** a single volume metric; no quality or concentration measure.
- **Worked example:** Worked example 5.

FW-C19-08 · The review-policy triangle · Original

- **Inputs:** a platform that reviews submissions.
- **Outputs:** targets for speed (time to decision), safety (harm prevented) and developer trust (consistency, clarity, appeals), a tiered process, and the measures that show the balance.
- **Assumptions:** submissions can be sorted by risk.
- **Tradeoffs:** improving two usually costs the third.
- **Failure modes:** one blanket process; opaque rejections; no appeals; no measure of false rejections.
- **Worked example:** IQ-C19-05.

Steps: **Sort submissions by risk** → **Automate routine checks** → **Human review for high risk** → **Explain every rejection** → **Offer appeals** → **Measure speed, harm and false rejections.**

4. Answer architecture

The platform answer, for a 25-minute round

Step	Minutes	What to produce
Clarify the platform and the goal	3	What it is, the sides, the stage (launch, growth, maturity), the objective
Map the sides and the loop	5	The platform map: what each side gives and gets; the flywheel; the first constraint
Stage-specific plan	4	Cold start (density, seed, narrow) or growth or mature governance
Governance and economics	6	Rules and review; take rate or pricing with a response model; openness; developer experience
Risks and metrics	5	The ecosystem risk scan; platform health metrics with guardrails
Decision and sequence	2	The recommendation, first steps, what would change it

Pacing rules. If the question names the stage (cold start, a policy), spend the time there. If you run behind, keep the map, the main tradeoff and one metric. Treat questions as steering.

How to say it. Name the sides first: "There are three groups: developers, users and the platform. Each gives and gets..." State the tradeoff with a number where you can.

How to close. Two sentences: "I'd start by [first step], governed by [rule], measured by [primary metric] with guardrails on [guardrails]. The main ecosystem risk is [risk], and I'd change course if [signal]."

Type: grow a marketplace from nothing

Use the cold-start playbook with the density arithmetic.

Type: an API or developer product

Use the developer-experience funnel and stability; metrics: time to first call, production share.

Type: platform policy (review, fees, openness)

Use the governance dial and the review-policy triangle; model the effect on each side.

Type: a key participant becomes a competitor

Use the risk scan: dependency, conflict of interest, trust; options and safeguards.

Type: platform metrics

Primary metric plus sides plus guardrails; include concentration.

Type: the short version (10 minutes)

Sides, the loop, the top tradeoff, one number. Protect the tradeoff.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"Who are the sides?"	You named one	Map each side's gives, gets and fears
"Why will it take off?"	The network-effect label	Name the mechanism, its locality and saturation
"How many sellers do you need?"	Density	Use the match probability and a threshold
"Developers say it's hard to start."	Developer experience	Funnel and time to first call
"What if they leave?"	Dependency and multi-homing	Concentration, switching costs, incentives
"What does the platform charge?"	Take rate	Model transaction value response; tiering
"What about regulators?"	Regulatory risk	Name exposure and design with it; mark facts to verify

5. Worked examples

All five examples are **Illustrative**: the products are fictional or use a product type as the setting only, and every figure is an invented assumption. The arithmetic was computed. Each is written as notes you could say aloud.

Worked example 1 — A payments API: improving the developer's first mile (Illustrative)

Setting: a payments API of the kind Stripe offers. No fact about any real company is claimed.

The situation. Developers say integration is painful. The PM starts by measuring the funnel (FW-C19-04). Of 10,000 developers who sign up in a month (invented): 7,600 get credentials (76%), 5,320 make a first successful call (70% of those), and 3,000 reach production (56% of those). Overall, 30% of sign-ups ship. The median time from sign-up to first successful call is 4 hours.

Find the friction at each step. Session recordings and support tickets (read by the PM and an engineer): at credentials, developers confuse test and live keys; at the first call, the quick-start example uses an outdated parameter, so the first request returns an error that says only "invalid request"; at production, developers do not know what to test before going live and hit surprise review steps.

Fixes, in order of effect. (1) Make the first example work as written and test it on every release. (2) Error messages that say what is wrong and how to fix it, with a link ("The 'amount' field must be an integer in the smallest currency unit; see the guide"). (3) Label test and live keys clearly and default to test. (4) A go-live checklist and a sandbox that mimics production. (5) SDKs in the languages most developers use.

What improvement looks like. If the three steps rise to 82%, 85% and 72%, the end-to-end share is $10,000 \times 0.82 \times 0.85 \times 0.72 = 5,018$, or 50%, up from 30%. The median time to first call falls to under an hour as the target (invented). The metric to own is time to first successful call, with the share reaching production as the outcome.

Stability as a feature. Developers build on the interface, so changes must not break them: version the API, announce deprecations months ahead, and support old versions for a published period. Breaking changes without notice would cost more trust than any documentation gain.

Guardrails. Support tickets per developer should fall; API error rates for correct requests should not rise; fraud and abuse checks must not slow legitimate developers.

What this shows. Treating the API as a product with a funnel found specific, fixable friction, and the metric (time to first call) pointed to the first mile.

Worked example 2 — An openness decision for a mobile operating system (Illustrative)

Setting: a mobile operating system platform of the kind Google develops. No fact about any real company is claimed.

The question. The platform owner licenses its operating system to many device makers. How much freedom should device makers have to modify it?

Sides (FW-C19-01). *Device makers* want freedom to differentiate and low cost. *Developers* want apps to run the same on every device. *Users* want a safe, consistent, secure device. *The platform owner* wants wide adoption, a healthy app ecosystem and a way to fund the work.

Options. (A) Fully open: any modification, no requirements. (B) A compatibility program: device makers may customize, but must pass tests of compatibility and security to use the platform's brand, services and app store. (C) Closed: only the platform owner makes devices.

Scoring (FW-C19-05). Criteria and weights (invented): partner adoption 25%, developer consistency 25%, security 20%, innovation 15%, control and revenue 15%. Scores 1 to 5:

Option	Adoption	Consistency	Security	Innovation	Control	Weighted
Open	5	2	2	5	1	3.05

Option	Adoption	Consistency	Security	Innovation	Control	Weighted
Compatibility program	4	4	4	4	3	3.85
Closed	2	5	5	2	5	3.80

Flip test. Weight adoption more (35%, 20%, 15%, 15%, 15%): open 3.35, compatibility 3.85, closed 3.50: compatibility stays first. Weight consistency and security more (15%, 30%, 30%, 10%, 15%): open 2.60, compatibility 3.85, closed 4.25: closed wins. So the compatibility program is the robust choice unless consistency and security dominate, and that is exactly the question to settle with the leadership: how much do we value reach against uniform quality?

Details of the middle path. What must devices pass (compatibility tests, security updates for a stated period)? What do device makers get in return (the brand, services, store access)? What happens to a device maker that fails (a path to fix before sanctions)? How are updates delivered to the long tail of devices? These are the governance decisions that make the middle path real.

Risks. Fragmentation if tests are weak; resentment if the program is arbitrary; regulatory scrutiny if the program is seen as tying services to terms [V: Apple Form 10-Q, 2026, on US and EU proceedings over app-store terms].

Recommendation. The compatibility program, with transparent tests, a published timeline for updates, and a review point at which the balance is revisited with data on fragmentation, security incidents and partner satisfaction. What would change my mind: a security crisis (move toward closed), or a collapse in partner adoption (move toward open).

What this shows. A weighted score with a flip test showed which choice was robust and what the real debate was about.

Worked example 3 — Setting a take rate for an enterprise marketplace (Illustrative)

Setting: a marketplace in an enterprise software suite of the kind Microsoft offers. No fact about any real company is claimed.

The situation. A marketplace lets independent software vendors sell add-ons to the suite's customers. The PM must propose a take rate. Operating the marketplace (payments, security review, support, hosting) costs about 8% of the value of transactions (invented).

How participation responds (invented). At a 20% take rate, the value of transactions through the marketplace is 100 (index). At 15%, more vendors list and prices are lower: 108. At 10%, 118. At 5%, 125.

The economics (FW-C19-05).

Take rate	Transaction value	Revenue	Operating cost (8% of value)	Contribution
20%	100	20.0	8.0	12.0

Take rate	Transaction value	Revenue	Operating cost (8% of value)	Contribution
15%	108	16.2	8.6	7.6
10%	118	11.8	9.4	2.4
5%	125	6.3	10.0	-3.8

Reading it. Revenue and contribution fall as the rate falls, since the increase in transaction value does not make up the lost share. Below about 8% the marketplace loses money on each transaction. A lower rate is justified only if it buys something the numbers above do not capture: faster ecosystem growth that makes the core suite more valuable (the platform earns from the suite, not only from the marketplace), a defense against competing marketplaces, or regulatory pressure.

The strategic view. If the marketplace's role is to make the suite stickier and more valuable, its contribution should be judged together with the suite's revenue and retention, not alone. Suppose the add-ons raise suite retention enough to be worth 10 (index) at a 10% rate; then the 10% marketplace is worth $2.4 + 10 = 12.4$, comparable with the 12.0 at 20%. That is an assumption to test, not a fact.

Tiering. A common design: a standard rate (say 20%), a reduced rate for small vendors or for vendors that use the platform's billing and co-selling (say 10%), and a transition period for any change. This protects small vendors, rewards deeper integration and avoids a cliff.

Recommendation. Start at 20% with a 10% tier for small vendors and co-sold sales, measure vendor growth and suite retention for two quarters, and review. Communicate rules and changes with notice. What would change my mind: evidence that vendors are leaving for a rival marketplace, or that add-ons lift suite retention by more than assumed.

What this shows. A small model made the floor (cost) and the trade (rate against participation) visible, and tied the decision to the platform's wider value.

Worked example 4 — The cold start: density before breadth (Illustrative)

The situation. A marketplace connects students with tutors who can come at short notice. The team has recruited 600 tutors and is deciding where to launch. A request is matched if at least one nearby tutor is available and accepts; suppose each tutor does so with probability 5% for a given request (invented).

The math (FW-C19-03). The chance of a match with N tutors is $1 - 0.95^N$: 20 tutors give 64%; 40 give 87%; 45 give 90%; 60 give 95%. To reach a 90% match rate, a market needs at least 45 tutors ($\ln 0.1 \div \ln 0.95 \approx 44.9$).

Two plans. *Spread:* launch in 30 cities with 20 tutors each. Each city has a 64% match rate. More than a third of requests fail, and students who fail once rarely return. *Concentrate:* launch in 13 cities with 45 tutors each ($600 \div 45 = 13.3$). Each city has a 90% match rate. The second plan gives a product that works where it launches, with word of mouth that compounds.

Seeding and single-player value. To recruit supply before demand exists, offer tutors something useful on their own (a scheduling and payment tool, a guaranteed minimum for the first months). Narrow further by subject or by school to raise the chance of a match for the students who come first.

What to measure. Not sign-ups, but liquidity: match rate, time to match, and the share of students who book a second session. Set a gate: expand to a new city only when the first reaches a 90% match rate and a repeat rate above a threshold.

Risks. Tutors and students may transact off the platform after the first match (disintermediation); mitigate with value they cannot get elsewhere (payment protection, scheduling, reviews). Quality and safety (verification of tutors) must be in place before launch, since one bad incident is costly.

What this shows. The arithmetic of matching turned "launch everywhere" into "launch where density is achievable," and gave a gate for expansion.

Worked example 5 — Reading platform health metrics (Illustrative)

The situation. A marketplace reports a monthly review (invented). Orders rose from 120,000 to 127,000 (+5.8%), at an average value of \$35, so transaction value rose from \$4.20 million to \$4.45 million. The leadership is pleased.

The full table (FW-C19-07).

Metric	Last month	This month	Reading
Orders	120,000	127,000	Up 5.8%
Active sellers	5,000	5,100	Up 2%
Active buyers	80,000	83,200	Up 4%
Match rate (requests that lead to an order)	62%	58%	Down 4 points
Time to first response	11 min	16 min	Slower
Disputes per 1,000 orders	8	11	Worse
Share of orders from the top 10 sellers	36%	40%	More concentrated
Take rate	14%	14%	Flat

Reading it. Growth is real in volume. But demand grew faster than supply (buyers +4%, sellers +2%), the match rate fell by 4 points, response time lengthened by 5 minutes, disputes rose by more than a third, and orders concentrated further in a few sellers. The platform is growing by pushing more buyers into a supply base that is not keeping up: a pattern that tends to precede lower repeat rates and seller churn. The transaction-value headline hides it.

What I'd do. Make match rate (successful transactions per request) the primary metric, with disputes, time to response and concentration as guardrails. Investigate the dispute rise (a new seller group? a category?). Boost supply where the match rate is lowest. Watch repeat purchase and seller churn next month for confirmation.

What this shows. A set of metrics across sides, with a primary metric tied to core value, showed a problem the volume number concealed.

6. Baseline, Strong, Standout

Prompt: "Design the growth plan for a new marketplace that connects home cooks with local diners."

Baseline answer

"It's a marketplace, so network effects will make it grow. I'd launch in as many cities as possible, spend on marketing to attract diners and cooks, and add features like reviews and a loyalty program."

What is wrong: "network effects" is a label; the plan spreads thin; no sense of what a successful match is, no sides, no risks.

Strong answer

"I'd map the two sides: cooks, who need a steady stream of diners and low hassle, and diners, who need reliable, safe, good food nearby. The cold start is the problem, so I'd launch in one neighborhood and seed cooks first, because without food diners leave. I'd measure liquidity: the share of diner requests that find a meal they want within a set time.

"I'd narrow the offering (a cuisine or a time of day) so that a few cooks can satisfy enough demand. Trust and safety matter in food: verification, food-safety rules and reviews. For growth, expand to the next neighborhood only when the first reaches a target match rate and repeat rate."

Standout answer

Everything in the Strong answer, plus:

"I'd put numbers on density: if each cook can satisfy a given request 5% of the time, I need about 45 cooks in a market to get a 90% match rate, so 600 cooks should go into about 13 markets, not 30. I'd give cooks a reason to join before diners come: a tool for taking orders and payments, and a guaranteed minimum for the first months.

"I'd think about disintermediation: after a first meal, diner and cook can deal directly, so the platform must offer something continuing (payment protection, scheduling, insurance, reviews). I'd model the take rate against that risk.

"I'd check multi-homing: cooks can also sell through other channels, so loyalty comes from a steady flow of orders and low friction. Legal exposure is real (food-safety rules vary by place), so I'd involve legal early and pick the first market for its rules.

"Metrics: primary, successful meals per request; guardrails, food-safety incidents, disputes, time to match, and the share of orders from the top cooks. What would change my plan: a first neighborhood that reaches the match target slowly (then the offering is wrong) or a safety incident (pause and fix)."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Maps the sides and focuses on one market	Computes the density needed and plans the number of markets
Seeds the hard side and measures liquidity	Addresses disintermediation, multi-homing and the take rate
Names trust and safety	Names legal exposure, guardrails and what would change the plan

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10] .

Company lens: Apple [as of 2026-10]. Apple runs an app store with published review guidelines and has changed its commission structure over time, including a reduced rate for small developers announced in 2020 Public [V: Apple press release of 18 November 2020; App Review Guidelines on Apple's developer site] . Third-party guides report that platform prompts at Apple concern review policy and the balance among developers, users and the platform Inferred . Practical consequence: weigh speed, safety and developer trust together, discuss second-order effects on security and quality, and label regulatory facts as to be verified.

Company lens: Google [as of 2026-10]. Google develops a widely licensed mobile operating system and runs an app store with its own policies and a reduced fee tier on the first \$1 million of developer revenue Public [V: Google Android Developers Blog, 16 March 2021] . Candidates commonly report platform prompts on openness and ecosystem health Inferred . Practical consequence: discuss compatibility and fragmentation, partners and developers together, and the tradeoff between openness and security.

Company lens: Microsoft [as of 2026-10]. Microsoft's platforms include operating systems, cloud and productivity suites with partner programs and marketplaces Public [V: Microsoft partner site: AI Cloud Partner Program; Microsoft Marketplace (AppSource and Azure Marketplace combined); posts to May 2026]. Candidates commonly report enterprise-platform prompts on partner programs and revenue sharing Inferred. Practical consequence: consider buyers, administrators and partners, tie marketplace economics to the suite's wider value, and mind compatibility with existing customers.

Company lens: Amazon [as of 2026-10]. Amazon operates a large third-party marketplace and a cloud platform Public [V: European Commission decision on Amazon Marketplace; AWS]. Candidates commonly report prompts on marketplace quality, seller trust and the tension of selling its own products alongside sellers' Inferred. Practical consequence: address seller trust, rules on use of data and placement, and measure the health of both sides.

Company lens: Meta [as of 2026-10]. Meta restricted third-party access to its platform data in 2018 after a data-misuse controversy, an example of governance tightening in response to risk Public [V: Facebook announcement of 4 April 2018 by Mike Schroepfer, as reported]. Candidates commonly report prompts on developer access, privacy and platform policy Inferred. Practical consequence: weigh developer value against user privacy and misuse, and describe how access is tiered, audited and revoked.

Company lens: Netflix [as of 2026-10]. Netflix is primarily a content service, not a two-sided platform, though it publishes open-source tools for other engineers Public [V: Netflix Tech Blog posts on experimentation and resilience testing]. Practical consequence: if a platform question concerns Netflix, check whether the platform lens fits before using it.

Company lens: Stripe [as of 2026-10]. Stripe's products are APIs for payments and related services, and it is widely described as emphasizing developer experience and documentation, and documents API versioning Public [V: Stripe API versioning documentation and engineering blog]. Candidates report developer-platform prompts Inferred. Practical consequence: lead with the first integration, time to first call, error messages and stability.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	"Network effects" as a label	No mechanism, locality or saturation	Name the mechanism and its limits
2	Designing for one side	Only buyers or only developers	Map every side's gives, gets and fears
3	Launching broad	Many markets, thin supply	Compute the density threshold; concentrate
4	Counting sign-ups	Growth measured by registrations	Measure liquidity: successful transactions
5	API as plumbing	No funnel, no time-to-first-call	Treat the first integration as the product
6	Breaking changes without notice	Developers surprised	Version, deprecate with notice
7	One blanket review process	All submissions treated alike	Tier by risk; explain; allow appeals
8	Take rate by revenue alone	Rate cut or raised without a response model	Model participation and the cost floor
9	Ignoring disintermediation	Value is only the introduction	Provide continuing value
10	One volume metric	GMV only	Add quality, concentration and developer metrics

9. Mental models applied

Four models, each tied to a decision in this chapter. `[[inversion]]` is from C02.

Metcalfe's law, and its limits (MM-metcalfe). The idea that a network's value grows with the square of its users [V: Metcalfe's law, as listed in software-engineering law references] is a useful intuition for why large networks are attractive, and a poor guide to planning: value depends on who can connect to whom, networks are often local, and value per user can saturate. Use it to explain why scale matters and never to forecast.

The tragedy of the commons (MM-commons). A shared resource that anyone can use without cost tends to be degraded by overuse [V: Hardin, Science 162(3859), 13 December 1968, pp. 1243 to 1248; the idea goes back to William Forster Lloyd, 1833]. An open platform with no quality bar can be filled with low-quality or abusive participants who each benefit and together harm everyone. Governance (review, ratings, fees, enforcement) is the platform's answer.

Postel's law (MM-postel). Be conservative in what you send and liberal in what you accept [V: Postel's law (robustness principle), as quoted in several sources]. For API design: send clean, well-specified responses, and tolerate reasonable variation in what developers send, with clear errors when you cannot. It is a basis for developer-friendly interfaces and also a caution: leniency can lock in sloppy behavior that is hard to change later.

Inversion (MM-inversion, see [[inversion]], C02). To design governance, ask how you would destroy the platform: make developers distrust it (arbitrary rules, sudden fee changes), flood it with low-quality supply, or let the largest participants take over. The list is the checklist of what to protect.

Engineering, design and data-science perspectives

- **Engineering.** Platform engineers own reliability, rate limits, versioning and security, and know what compatibility costs. Ask what breaking changes are in the pipeline and how they will be handled; ask for the error messages to be reviewed as product text.
- **Design and research.** Research on both sides: sit with developers integrating for the first time; interview sellers and buyers. Design the seller or developer console as a product with its own users.
- **Data science.** Two-sided markets complicate measurement: tests of a change on one side can affect the other (interference, C14.4), so market-level or cluster designs may be needed. Ask for liquidity measures and concentration measures, and for dashboards by side.
- **Policy, legal and trust and safety.** Rules on fees, openness, competition and data are changing [as of 2026-10]; safety teams define what must be blocked. Involve them early in any change to rules, fees or access.

Checklist: before you recommend a platform move

1. Have I mapped every side's gives, gets, alternatives and fears?
2. Did I name the network-effect mechanism, its locality and its saturation point?
3. For a new platform, did I compute the density needed and choose where to start?
4. Did I treat the API as a product, with a funnel and a stability policy?
5. Did I set governance as a three-way tradeoff (speed, safety, trust) and model the take rate against participation and cost?
6. Did I scan for dependency, concentration, disintermediation, conflict with participants and regulatory attention?
7. Is the primary metric tied to successful transactions, with guardrails across sides?
8. Did I say what would change my recommendation?

10. Case studies

CS-C19-01 · The marketplace spread across eight cities · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A marketplace connecting households with cleaners has raised money and recruited 160 cleaners. To show reach, the team launches in eight cities at once, with 20 cleaners in each. After three months, the team sees many sign-ups by households and a low repeat rate.

Constraints. A fixed supply of 160 cleaners, a marketing budget, and a board expecting national coverage.

Data. For a household request, suppose each cleaner in the city can take the job with probability 4%. Of households that tried a booking in the first month, 44% did not get a match within a day (invented), and of those, 85% never tried again. Households that were matched rebooked at a healthy rate.

Problem definition. As an observable problem: "Households in each city often request a cleaning and find nobody available, and they do not come back."

Analysis. With 20 cleaners per city, the match chance is $1 - 0.96^{20} = 56\%$, consistent with the 44% failure the team saw. A sign-up metric counted the failures as growth. If the same 160 cleaners were in 2 cities (80 each), the match chance would be $1 - 0.96^{80} = 96\%$. Density per market, not the total, determines whether a household's first experience works, and the first experience determines repeat use. The cold-start playbook (FW-C19-03) says: narrow the market, seed the hard side, measure liquidity, expand when the gate is met.

Options.

Option	Optimizes	Costs
A. Keep eight cities and add marketing	Apparent reach	Most requests fail; wasted spend
B. Concentrate in two cities; pause the others with a waitlist	Match rate and repeat use	Disappoint six cities; board optics
C. Recruit 500 more cleaners	Density everywhere	Cost and time; hiring quality risk

Tradeoffs. B gets the product working with the supply in hand and builds proof; C is slower and costly; A repeats the failure.

Decision. Choose B. Put all 160 cleaners into the two densest cities. Primary metric: match rate within a day (target 90%); guardrails: time to match, cleaner earnings (so they stay), complaints; gate: expand to a third city when two cities have a 90% match rate and 30-day repeat above a threshold. Tell the board the plan and the gate.

Outcome. In this fictional scenario the match rate in the two cities rose to 95%, repeat bookings rose, and cleaners' earnings increased because they had more jobs (invented). The third and fourth cities launched four months later with 50 cleaners each, after recruiting.

Alternative outcomes. If the two cities had been poor for demand, a match rate of 95% with few requests would have shown that supply was not the constraint, and the team would have changed cities or marketing. If cleaners had left because of low earnings in thin markets, the earnings guardrail would have shown it. The lesson: density per market is the unit of progress, and a measure of liquidity is the honest dashboard.

CS-C19-02 · A mobile app store as a platform, reconstructed from public documents · Reconstructed · Advanced

Sources (types): Apple's press releases (the App Store's first-weekend announcement of 14 July 2008 and the App Store Small Business Program of 18 November 2020, with Apple's feature of the same day), primary sources for its statements and figures; Apple's published App Review Guidelines, read at the level of their structure and not quoted; Google's announcement of its reduced service fee (Android Developers Blog, 16 March 2021), a primary source for its statement; Apple's Form 10-Q for the quarter ended June 2026 (SEC), for the status of fee disputes; and reference summaries and press coverage for the launch day, the 70/30 split and the launch-day count of about 500 applications. The analysis is an inference from these public documents, labeled throughout, and is not based on internal data, deliberations or financials. Re-read in October 2026.

Situation. The App Store launched on 10 July 2008 with roughly 500 applications, according to the count Steve Jobs gave on launch day as reported by the press, and a revenue split in which developers kept 70% of sales revenue and Apple kept 30% [V: reference summaries quoting Apple's 2008 announcements]. On 14 July 2008 Apple announced that users had downloaded more than 10 million applications in the first three days and that more than 800 were available [V: Apple press release, 14 July 2008]. As the catalog grew, discovery plausibly became a problem for developers; that is an inference from the scale (1.8 million apps by 2020), not a sourced statement [inference]. In November 2020, Apple announced a program, launching on 1 January 2021, under which developers who earned up to \$1 million in proceeds (defined as post-commission earnings) in the previous calendar year would pay a reduced 15% commission, with requalification possible in later years, while the standard 30% remained for those above that level; in a feature published the same day Apple said that half a billion visitors a week engaged with 1.8 million apps [V: Apple, 18 November 2020]. Google announced on 16 March 2021 that from 1 July 2021 its Play service fee would be 15% on the first \$1 million of revenue each developer earns each year, available to every developer using Play billing and returning to 30% beyond that amount [V: Android Developers Blog, 16 March 2021; TechCrunch]. Apple publishes App Review Guidelines arranged, in its words, into five sections (Safety, Performance, Business, Design and Legal), with notes on what to do before and after submitting [V: Apple App Review Guidelines].

Constraints. A two-sided and developer platform: developers supply apps and need distribution, payments and trust; users need discovery, safety and ease; the platform sets the rules, funds itself through a commission and benefits from the ecosystem's effect on its devices. Public criticism and legal and regulatory scrutiny of fees and rules have been widely reported: in its filing for the quarter ended June 2026, Apple described a US court order barring it from charging commission on purchases made outside an app, an appeals court ruling that some commission may be charged on such link-out purchases with the case sent back to the district court, the Supreme Court's grant on 30 June 2026 of Apple's petition to

review the legal standard for civil contempt, and European Commission investigations under the Digital Markets Act [V: Apple Form 10-Q for the quarter ended 27 June 2026]; the facts of specific disputes beyond that are outside the sources used here [as of 2026-10].

Data. As reported: the dates, the revenue split, the first-weekend downloads, the later growth to 1.8 million apps, the 2020 small-business tier and its terms, the Google tier, and the discovery problem as the catalog grew. No internal data is used.

Problem definition. *The author's reading.* The platform had to design a set of rules and prices that would attract developers, protect users and fund itself, while keeping a catalog discoverable as it grew. As an observable problem: "Set the rules, the review and the commission so that developers keep building, users trust what they download, and the platform is paid for what it provides."

Analysis. *The author's analysis, as inference from public documents.*

Sides and loop (FW-C19-01). Users buy devices partly for the apps; developers build for the devices because users are there and can pay; each side raises the value to the other, a cross-side effect that is local to the platform's own users and strengthened by the integration of device, software and store (an ecosystem). Developers can and do multi-home across platforms, so the platform has to keep the best terms of distribution, tools and reach. Inference: the early 70/30 split was a simple, attractive offer compared with the earlier mobile software market, which contemporary press described as controlled by carriers, which took 40% to 50% of revenue on apps distributed through their phones against Apple's 30% [V: TechCrunch, 24 July 2008].

Governance (FW-C19-05, FW-C19-08). A review before listing protects users and the platform's reputation (safety, privacy, quality) and imposes delay and uncertainty on developers, which affects developer trust. The public guidelines and the scale of the catalog imply a tiered process in which automated checks handle routine submissions and humans handle harder cases; this is inference, since the internal process is not described in the sources used here.

Take rate. A flat commission is simple and predictable; its level is a lever with effects on developers' margins and prices. The 2020 small-business tier, by inference, targeted the many small developers (Apple said the program would benefit the vast majority of developers who sell digital goods and services [V: Apple, 18 November 2020]) in a way that cost relatively little revenue while improving the platform's standing with developers and in the regulatory conversation; Apple itself framed it as helping small businesses. Google's later tier is consistent with competitive and similar regulatory pressure, again an inference; it is also structured differently, since it applies to the first \$1 million each developer earns each year with 30% beyond that, whereas Apple's tier depended on the previous year's proceeds [V: Android Developers Blog; Apple, 18 November 2020].

Ecosystem risk (FW-C19-06). Dependency: developers who depend on the store for distribution are exposed to rule changes, and the platform depends on a few large developers for much of its revenue (concentration, a general feature of such stores). Disintermediation: where an app can sell digital goods through its own payment system, the platform's commission is at risk; rules on payment and on linking out are the subject of

public disputes [V: Apple Form 10-Q, 2026]. Conflict with participants: the platform's own apps compete with third-party apps, and developers' concern about placement and data use is widely reported. Regulatory attention: the fee and openness rules draw scrutiny in several jurisdictions. These risks are public topics; the case does not assess the merits of any dispute.

What the evidence does not show. The platform's costs, margins or internal decision criteria; the effects of the 2020 tier on revenue; and how developers' behavior changed. The case supports a structural analysis only.

Options. *The author's reconstruction of choices a platform owner could face; not a record of any company's deliberations.*

Option	Optimizes	Costs
A. Keep a single flat commission and strict review	Simplicity; consistency; revenue	Developer dissatisfaction; regulatory exposure
B. Tier the commission (lower for small developers) and keep strict review	Developer goodwill; modest revenue cost	Complexity; large developers unaffected
C. Open alternative distribution and payment with safeguards	Developer choice; regulatory fit	Security and quality risk; revenue; consistency

Tradeoffs. A is simple and draws criticism; B responds to small developers cheaply; C changes the platform's model and brings new risks. The choice among them depends on how much the platform's value rests on control and consistency, and on what regulators require.

Decision. As documented, the platform launched with review and a 70/30 split, and in 2020 introduced a 15% tier for small developers.

Outcome. Reported: growth to 1.8 million apps and half a billion weekly visitors by late 2020 (Apple). Later developments are outside the sources used here.

Alternative outcomes. *Inference.* A looser review might have grown the catalog faster and increased the presence of harmful or low-quality apps, with costs to user trust. A lower initial commission might have attracted more developers and reduced the platform's funding for its own services. A reader should keep in mind that a platform's rules are hard to change once many businesses depend on them, which is why governance decisions are among the most consequential and why they deserve the three-way tradeoff analysis of this chapter.

11. Interview questions and model answers

Seven questions: two Beginner, two Intermediate, three Hard. Model answers are **Illustrative** (composite candidates), every figure is an assumption, and the arithmetic was computed. Each is written against the six behaviors in section 1. Where a real company's product is the setting, the answer reasons from public information and states no internal fact. The answers are fuller than a first pass in a real round.

IQ-C19-01 · Beginner · strategy · ST

Prompt. What is a platform? How is a two-sided market different from a developer platform?

Rubric.

- **Baseline:** "A platform is a big app that lots of people use."
- **Strong:** Defines a platform by multiple participant groups and gives both types with examples.
- **Standout:** Names the different core jobs and hard problems of each, notes hybrids, and says when the label does not apply.

Model answer. A platform is a product that others build on or transact through, which creates value for more than one group. The defining feature is that the value to one group depends on another group's participation.

A *two-sided market* has two groups who need each other: riders and drivers, buyers and sellers. The platform's core job is matching and trust. The hard problems are the cold start (neither side comes until the other is there), keeping enough supply near the demand (liquidity), and disintermediation (participants bypassing the platform after a match). The primary metric is successful transactions.

A *developer platform* provides a foundation that others build products on, through APIs and tools: an operating system, a cloud service, a payments API. The core job is to be reliable, well-documented and stable. The hard problems are developer experience (how fast a developer succeeds), stability (not breaking what people built), and governance. The primary metric is developers who ship to production, or what they build.

Many platforms are both: an app store has developers (who build) and users (who buy), so it faces review-policy, fee and discovery problems on top of the two kinds of problem.

When it does not apply: a product with many users but no interaction among them and no outside builders is just a popular product. Calling it a platform imports the wrong questions. My first step with a platform question is to name the sides and what each gives and gets.

Follow-ups.

- **Q.** Give an example of a hybrid, and say when the platform label does not apply. **A.** "An app store is both: a two-sided market between users and developers, and a developer platform with tools, interfaces and rules. The label does not apply when no group's value depends on another group's participation. A company that offers an API to a few customers but whose product works the same whether or not anyone builds on it is a product with an interface, not a platform. The test I use is whether adding more participants of one kind makes the product more valuable to another kind."
- **Q.** What are the hard problems of each type? **A.** "For a two-sided market: the cold start (each side waits for the other), liquidity (the chance a buyer finds a seller and a seller finds work), and trust between strangers. For a developer platform: interface stability so that builders can rely on it, a good developer experience (time to a first working call), and governance of what builders may do. The measures differ accordingly: liquidity and

repeat transactions for the market, production-active developers and time to first call for the developer platform."

IQ-C19-02 · Beginner · strategy · ST

Prompt. A freelance marketplace has 5,000 freelancers and 300 clients. Clients complain they cannot find the right person, and freelancers complain they get no work. What's going on, and what do you do?

Rubric.

- **Baseline:** "Get more clients with marketing."
- **Strong:** Diagnoses a supply-demand imbalance and a matching problem, and proposes narrowing.
- **Standout:** Quantifies the ratio, uses liquidity, separates the two complaints, and proposes seeding demand and curating supply, with a metric.

Model answer. The two complaints are one problem seen from each side. There are about 17 freelancers for each client ($5,000 \div 300$), so work is scarce for freelancers, and with so many to choose among and no structure, clients struggle to find the right person. The platform has too much unfocused supply and too little demand.

Diagnose by segment. Not all freelancers compete for the same work. Suppose most are in a few popular categories while demand is concentrated in a few others (invented). Look at the match rate and time to hire by category: some categories may be healthy, others empty on one side.

What I'd do. (1) *Narrow:* focus on the categories where demand exists and where the platform can reach density, and stop recruiting supply in saturated ones. (2) *Raise the quality of matching for clients:* better search, a short brief that routes to a small set of qualified candidates, so clients see a few strong options and not a thousand. (3) *Grow demand where supply is strong:* outbound to clients who need those skills. (4) *Curate supply:* verified skills and reviews, so clients can trust what they see and the best freelancers get the work, which keeps them.

Metric. Not sign-ups. Primary: the share of client briefs that result in a hire within a week (hire rate), with time to first qualified proposal, and freelancer earnings for the top and median freelancer as guardrails (if only a handful earn, supply will leave). I'd check that clients rehire.

Risk. Cutting supply feels like shrinking; the aim is balance. What would change my mind: if clients' real issue is trust and not selection, in which case verification and protection matter more than matching.

Follow-ups.

- **Q.** How would you quantify the imbalance and segment it? **A.** "The ratio is about 17 freelancers per client. Then I would segment: if 60% of freelancers are designers and most client projects are for developers, the imbalance is concentrated in design. I would measure, by category, projects posted, proposals per project (for example 85 in design

against 12 in development), the share of projects filled within seven days, and the share of freelancers who won any work in 30 days. This shows where supply is idle and where demand is unserved, and it turns two complaints into one table."

- **Q.** What would you do to seed demand and curate supply? **A.** "Pause freelancer onboarding in the oversupplied categories and focus on bringing in clients through outbound outreach and partnerships, since demand is the constraint. For clients, replace an open list with a matched shortlist of five and a quick way to invite them. The measure is the share of projects filled within seven days and the shortlist acceptance rate, with freelancer earnings per active freelancer as a guardrail. Reopen supply onboarding in a category only when proposals per project fall below a set level, for example 25."

IQ-C19-03 · Intermediate · strategy · ST · Stripe

Prompt. A payments API team can ship a cleaner design that simplifies integration for new developers but breaks 3% of existing integrations. Do you ship it?

Rubric.

- **Baseline:** "Yes, simpler is better," or "No, never break developers."
- **Strong:** Weighs benefits for new developers against costs to existing ones, and considers versioning.
- **Standout:** Quantifies both, proposes a non-breaking alternative (a new version alongside the old), a deprecation timeline and migration support, and treats trust as an asset.

Model answer. The setting is general: I know such products only from public information, and nothing here describes any real plan. I'd not ship it as a breaking change on a flag day. I'd look for a way to get the benefit without the break.

Numbers (invented). 200,000 businesses have integrated; 3% is 6,000. Helping each migrate costs about \$150 of support and engineering time: \$900,000 once. The benefit: a simpler API raises the share of new developers who reach production from 56% to 62% (a better first mile, C19.3); with 20,000 new developers a year, that is 1,200 more integrations, each worth \$2,000 a year in revenue (invented): \$2.4 million a year. The risk: if 5% of the 6,000 affected businesses leave because of the disruption, 300 integrations are lost at \$2,000: \$600,000 a year. Net in year one: \$2.4 million – \$0.9 million – \$0.6 million = +\$0.9 million. That looks positive, but it ignores trust: a platform known for breaking integrations raises the cost of building on it for everyone, and that is hard to measure and costly to lose.

Alternative. Ship the cleaner design as a new version, and keep the old working. New developers get the simple path; existing integrations are untouched. Cost: maintaining two versions for a while. Then deprecate the old version on a published schedule (say 24 months), with notice, a migration guide and tooling, and focus support on the businesses most affected. This captures most of the \$2.4 million without the \$0.6 million risk and the trust cost.

Decision. Introduce the new version, keep the old, announce a long deprecation path, and measure adoption of the new version by new and existing developers, support tickets, and churn among those who migrate. What would change my mind: if the old design carries a security flaw, a shorter timeline with strong support is justified.

Company note. Stripe is the setting only: Inferred from candidate reports of developer-platform prompts and Public that Stripe documents API versioning [V: Stripe API versioning documentation; the prompt descriptions are candidate-reported and unverified]. No Stripe data or plan is claimed.

Follow-ups.

- **Q.** What non-breaking alternative would you propose? **A.** "Ship the cleaner design as a new version alongside the old one, so existing integrations keep working. Offer migration tooling and a compatibility layer for the 6,000 affected integrations, announce a deprecation timeline of 18 to 24 months with reminders, and give the largest users direct help. New developers get the simple version by default. The old version stays supported until usage falls below a threshold, and I would measure the share migrated each quarter. This gets most of the benefit and protects the trust that developers place in the platform's stability."
- **Q.** How would you compute the case? **A.** "With invented figures. Helping 6,000 integrations migrate at about \$150 each costs \$900,000 once. Suppose the simpler design lifts activation of new developers from 40% to 48% of 20,000 new developers a year, 1,600 more integrations, each worth \$500 a year: \$800,000 a year. Payback is just over a year before counting the risk. The risk of a flag-day break is that 2% of the affected developers leave and others lose trust, which could cost far more than the migration. The numbers favor a staged migration over a break."

IQ-C19-04 · Intermediate · strategy · ST · Microsoft

Prompt. A software vendor accounts for 12% of your enterprise marketplace's revenue. It asks for a lower revenue share and a say in your roadmap. How do you respond?

Rubric.

- **Baseline:** "Give it what it wants, it's too big to lose," or "treat everyone the same."
- **Strong:** Weighs the vendor's value and leverage against fairness to other vendors, and offers structured options.
- **Standout:** Computes the cost, separates the economic request from the influence request, proposes volume tiers open to all and a partner council, and addresses concentration risk and the precedent.

Model answer. The setting is general: I know such platforms only from public information, and nothing here describes any real plan. Two requests are bundled and need separating: price and influence.

Price. Suppose the marketplace's revenue is \$50 million at a 20% share, so the vendor generates \$6 million in revenue, on \$30 million of sales. A cut to 12% would reduce revenue from that vendor to \$3.6 million, a loss of \$2.4 million a year. If other vendors learn of a

special deal, they will ask for it, and trust in the rules erodes. If the vendor really generates 12% of revenue, it is also a concentration risk: its departure would hurt, which is a reason to reduce dependence as well as to keep it.

Principled answer. Offer a published volume tier that any vendor can qualify for, with a lower rate above a sales threshold, perhaps with a requirement that gives the platform something in return (co-selling, deeper integration, support commitments). The vendor gets a better rate because of its size, and every other vendor can see the rule and aim for it.

Influence. A seat on a partner advisory council, where several vendors give input to the roadmap, is reasonable; a private veto is not, because it would trade the platform's neutrality and its other vendors' trust for one account. I'd listen to the vendor's needs (the real product problems) and consider them on the same criteria as other requests.

Concentration. Work on the broader base: grow the number of mid-sized vendors and measure the share of revenue from the top 10 (target a decline over time).

Decision. Offer the volume tier and the council, hold on a bespoke deal, and give the vendor a dated plan for roadmap items that serve many vendors. What would change my mind: if the vendor is truly strategic (it brings customers that others do not), a bigger commitment from them in exchange for a deeper partnership, structured as a tier open to peers.

Company note. Microsoft is the setting only: Inferred from candidate reports of enterprise partner and marketplace prompts at Microsoft [unverified: candidate-reported]. No Microsoft data or program is claimed.

Follow-ups.

- **Q.** How do you separate the price request from the influence request? **A.** "They are different asks with different answers. On price: if the vendor generates \$6 million at a 20% share on \$30 million of sales, a 12% share would cut that to \$3.6 million, a loss of \$2.4 million, about 5% of the marketplace's \$50 million revenue. I would not agree to a bespoke rate; I would offer volume tiers open to every vendor, for example 20% under \$10 million, 16% from \$10 to \$30 million and 14% above, so that the vendor benefits and the precedent is a rule. On influence: a partner council with several vendors that advises on the roadmap, with published criteria, while decisions stay with the platform."
- **Q.** How do you handle concentration risk and the precedent? **A.** "By reducing dependence and making terms public. Twelve percent of revenue from one vendor is a risk in both directions: the vendor could leave, and the marketplace could be pressed into terms that hurt others. I would diversify by growing other vendors, agree a multi-year contract with notice periods, and have a contingency if the vendor moves its sales elsewhere. Any special term given to one vendor becomes a demand from all, so terms should be rules, not deals. I would also track vendor concentration as a standing metric."

IQ-C19-05 · Hard · strategy · ST · Apple

Prompt. Design the review policy for an app store: speed versus safety versus developer trust.

Rubric.

- **Baseline:** "Review everything carefully," or "make it fast."
- **Strong:** Names the three goals and designs a tiered process.
- **Standout:** Sorts submissions by risk, quantifies reviewer capacity, builds explanations and appeals, measures false rejections and harmful approvals, and sets governance for changes to the rules.

Model answer. I know the topic only from public information; this describes a design for a store of the kind Apple and Google operate, not any real process. The three goals pull against each other (FW-C19-08). Strict review raises safety and costs speed and trust; light review raises speed and risk. The way through is to match scrutiny to risk.

Tier by risk. Suppose 100,000 submissions a week (invented). Low risk (about 70%): updates to established apps with no new permissions or content types. Medium (25%): new apps in ordinary categories and updates that add features. High (5%): new apps in sensitive areas (payments, children, health, financial products), apps requesting powerful permissions, and apps from new accounts with signals of abuse.

Process. Automated checks on everything (malware, policy violations, privacy declarations): minutes. Low risk: automated approval within an hour. Medium: automated checks plus a short human review, within 24 hours. High: deeper human review, two to three days, with specialists.

Capacity check. Human time: 25,000 medium submissions at 0.5 hours is 12,500 hours; 5,000 high at 3 hours is 15,000 hours; total 27,500 hours a week, about 786 reviewers at 35 productive hours each. That is the real cost, and why automation and good tiering matter: each point of submissions moved from medium to low saves hundreds of reviewer hours.

Developer trust. Published, specific rules with examples; consistent decisions (audit a sample for consistency); a reason with every rejection, tied to a rule; a fast appeals path with a different reviewer; a way to ask for guidance before submitting; predictable timing. Trust is lost mainly through opaque, inconsistent or slow decisions.

Safety. Post-release monitoring (reports, anomaly detection), fast takedown and a way to revoke developer access; stronger identity checks for developers who handle sensitive data.

Measures. Median and 95th-percentile time to decision by tier; harmful-approval rate (apps approved that later violate rules); false-rejection rate (rejections overturned on appeal), target under 1% of decisions; developer satisfaction; and user-reported harm. Guard against gaming: a faster median hides a long tail.

Governance of the rules. Changes announced with notice, a comment period for significant changes, and a review of rules against outcomes each quarter.

Decision. Tiered review with automation for the bulk, human depth where risk is high, explanations and appeals for trust, and the measures above. What would change my mind: a spike in harmful approvals (tighten the low tier) or a rise in overturned rejections (retrain reviewers or simplify rules). Regulatory requirements on app review differ by place and are changing [V: Apple Form 10-Q, 2026; EU Digital Markets Act].

Company note. Apple is the setting only: **Public** that Apple publishes review guidelines and **Inferred** from third-party reports that platform prompts at Apple include review policy [V: Apple App Review Guidelines on Apple's developer site; the prompt descriptions are candidate-reported and unverified]. No claim is made about any real review process, staffing or metrics.

Follow-ups.

- **Q.** How do you tier by risk and size reviewer capacity? **A.** "Three tiers. Low-risk updates (70% of 100,000 weekly submissions, invented) pass through automated checks with random sampling for audit; medium-risk submissions (25%) get a human review of about 8 minutes within 24 hours, which is $25,000 \times 8 \div 60 =$ about 3,300 hours a week, or about 83 reviewers at 40 hours; high-risk categories such as payments, children, health and finance (5%) get a 45-minute deep review: $5,000 \times 0.75 =$ 3,750 hours, about 94 reviewers. In total about 177 reviewers, so changing the tier shares changes the staffing. The numbers show where speed is bought and where safety is."
- **Q.** How do you measure false rejections and harmful approvals, and govern rule changes? **A.** "False rejections by the appeal overturn rate and an audit of a sample of rejections; harmful approvals by post-release incidents per 1,000 apps and by user reports, with a review of each serious case. Add time to decision by tier and developer satisfaction. For governance: rule changes are published with notice (for example 60 days), explained with the reason, discussed in a developer council, and recorded in a change log. Developers can predict what will pass, and the owner can show that rules apply to everyone, including its own apps."

IQ-C19-06 · Hard · strategy · ST

Prompt. A home-services marketplace suspects that customers and providers are booking repeat jobs off the platform. How do you detect and reduce this?

Rubric.

- **Baseline:** "Ban contact information sharing" or "cut the fee."
- **Strong:** Defines the problem, finds proxies to measure it, and offers value-based solutions.
- **Standout:** Quantifies the leakage, shows that a fee cut alone may not pay, designs continuing value, considers fairness to providers, and sets a test and guardrails.

Model answer. Disintermediation is likely where the platform's value is the introduction and repeat jobs are easy to arrange directly. I'd start by measuring how big it is.

Detection. Direct measurement is impossible, so use proxies: pairs of customer and provider with a first job and then no further bookings though the customer keeps using the category (the customer's other bookings continue with someone else, or provider availability shows busy); attempts to share contact details in messages; repeat-booking rates lower than in comparable categories; providers whose reviews mention working "outside the app."

Estimate leakage: suppose (invented) 100,000 first jobs a month, 40% of pairs would rebook, and 30% of those rebookings leak. With a 15% take rate on a \$60 job (\$9), leakage costs $100,000 \times 0.4 \times 0.3 \times \$9 = \$108,000$ a month.

Does cutting the fee on repeat jobs fix it? Suppose repeat jobs are 40,000 a month; today 70% stay on-platform: 28,000 at \$9 = \$252,000. If a lower rate of 8% (\$4.80) cut leakage from 30% to 10%, on-platform would be 36,000 at \$4.80 = \$172,800. Revenue falls. To keep revenue at \$252,000 at \$4.80 would need 52,500 repeat jobs on the platform, more than the 40,000 that exist. So a fee cut alone cannot pay; it must be paired with something else.

What works: continuing value. Things the customer and provider cannot easily replicate off-platform: payment protection and dispute resolution, insurance or guarantees on damage, scheduling and reminders, recurring bookings with automatic payment, a verified record of work and reviews that the provider builds and takes pride in, financing or faster payouts for providers. If a guarantee costs \$1 per job and cuts leakage from 30% to 20%, on-platform repeat jobs become 32,000 at $(\$9 - \$1) = \$256,000$, which slightly exceeds today's \$252,000 and builds trust.

Rules. Contact-sharing bans have limited effect and annoy honest users; use them lightly for the first booking only, and focus on incentives.

Test and guardrails. Pilot the guarantee and recurring-booking tools in two markets with a holdout; measure repeat rate on-platform, provider earnings, customer satisfaction and the proxy for leakage. What would change my mind: leakage proving far smaller than the proxies suggest, in which case the effort belongs elsewhere.

Follow-ups.

- **Q.** How do you detect leakage? **A.** "With proxies, since direct measurement is impossible. A customer and provider who had a first job and then no more bookings together while the customer keeps booking in the category with others; contact details in messages before booking; a lower repeat rate for pairs than in similar categories; and a short survey of providers and customers who stopped booking. I would estimate the leakage as the gap between the repeat rate we see and the rate in a comparable category where leakage is hard, and I would report it as a range."
- **Q.** Why might a fee cut alone not pay, and what continuing value would you add? **A.** "A cut from 15% to 10% lowers revenue on every job, including the ones that do not leak, and leaks may continue if the platform adds nothing after the first job. Instead add value that persists: payment protection and a guarantee that only apply to jobs booked on the platform, insurance, scheduling and reminders, a record of work and reviews, and one-tap rebooking with a small loyalty reward after the third job. Test it in one city for eight weeks and measure repeat bookings per pair and provider satisfaction, with provider earnings as a guardrail so that fairness is protected."

IQ-C19-07 · Hard · strategy · ST

Prompt. Define the metrics you would use to run a developer platform for a cloud API, including how you would tell early that something is going wrong.

Rubric.

- **Baseline:** "Number of developers and API calls."
- **Strong:** Defines a primary metric and metrics for acquisition, activation, usage, quality and revenue.
- **Standout:** Uses time to first call and production share, adds ecosystem health (concentration, churn), builds leading indicators and guardrails, and reads trends by cohort.

Model answer. I'd start from the platform's core value: developers building products that work and keep working. A volume metric such as API calls can rise while developer satisfaction and ecosystem health fall, so I'd use a primary metric and guardrails.

Primary metric. Developers whose integrations are live in production and active over the last 30 days (production-active developers). It reflects success, not interest.

The funnel (FW-C19-04). Sign-ups; the share who get credentials; the share who make a first successful call, with the median time to it; the share who reach production. Each stage is a place to look for friction; the first call time is the leading indicator of adoption.

Usage and value. API calls per production-active developer; the number of distinct endpoints used (breadth); revenue per developer, and the share of developers growing their usage.

Quality and trust. API availability and latency at the 95th and 99th percentiles; error rate for correct requests; support tickets per 1,000 developers and time to resolve; documentation feedback; the number of breaking-change incidents.

Ecosystem health. Concentration: the share of API volume or revenue from the top 10 developers; developer churn (production-active developers lost per month) by cohort; multi-homing (developers also using rivals) from surveys; the share of new developers coming from referrals.

Early warnings. (1) Time to first call lengthening for new cohorts, before sign-ups fall. (2) The production share of the latest sign-up cohort falling relative to earlier cohorts. (3) Rising error rates on a version before its deprecation. (4) Support ticket growth faster than developer growth. (5) Rising concentration, meaning growth is coming from a few large accounts. Read them by cohort, since averages blend old developers who are fine with new ones who are struggling (C13, C05).

Guardrails. No improvement in sign-ups counts if the production share falls; no change counts that raises the error rate.

How I'd act. A weekly dashboard by side and cohort, a threshold on each early warning with an owner, and a monthly review of developer interviews alongside the numbers. What would change the set: a platform with a very different developer type (enterprise integrators versus hobbyists) would need separate funnels.

Follow-ups.

- **Q.** What leading indicators show that something is going wrong? **A.** "Rising time to first successful call; a falling share of new integrations that reach production within 30 days;

rising error rates in new integrations; a falling documentation satisfaction score; growing concentration of traffic in a few developers; and rising support contacts per 1,000 developers. Any one can be noise, so I would watch the set and set alerts, for example if the production share falls by five points two months in a row. These move before API call volume does, which is why call volume alone is a poor early warning."

- **Q.** How do you read the trends by cohort? **A.** "By sign-up cohort. Compare the share of each quarter's new developers who are production-active at 90 days: if the first-quarter cohort reached 35% and the third-quarter cohort reached 22%, onboarding has regressed even if total active developers are growing because older cohorts are loyal. I would split by developer type (hobbyist, startup, enterprise) and by region, then look for the step where the new cohort stalls, whether it is the first call, the first deployment or billing set-up. Cohorts show the health of the pipeline, which totals hide."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked, and the arithmetic was computed.

EX-C19-01 · Classify the platform

Problem. Classify each as a two-sided market, a developer platform, a hybrid, or not a platform, with one reason: (a) a ride-hailing app; (b) a cloud computing service; (c) a mobile app store; (d) a single-player puzzle game with millions of users; (e) a freelance job board; (f) a video streaming service with licensed content.

Context. A warm-up.

Data. None.

Constraints. Four minutes.

Expected approach. FW-C19-02.

Solution. (a) Two-sided market: riders and drivers need each other. (b) Developer platform: others build on it. (c) Hybrid: developers build and users buy. (d) Not a platform: no outside builders or interaction among users. (e) Two-sided market: employers and candidates. (f) Mostly not a platform: users do not transact with one another or build on it, though it may have partner tools; check before applying the lens.

Alternative solution. Note that (a) also has developer features if it exposes an API for partners.

Tradeoffs. The label matters because it brings the right problems.

EX-C19-02 · Map the sides

Problem. Build a platform map for a freight marketplace connecting shippers and trucking companies: what each gives and gets, their alternatives and fears, and the first constraint.

Context. A new platform.

Data. None; reason.

Constraints. Six minutes.

Expected approach. FW-C19-01.

Solution. *Shippers* give loads and payment; get reliable capacity, price transparency, tracking; alternatives: brokers, existing carriers; fears: late or damaged freight, unreliable carriers. *Carriers* give trucks and driving; get full trucks (less empty running), faster payment; alternatives: brokers, regular customers; fears: low rates, slow payment, bad loads. *The platform* gives matching, payment, trust (verification, insurance), tracking. The value loop: more loads attract carriers, which lowers prices and raises reliability, which attracts shippers. The first constraint: lane density (loads and trucks on the same routes at the same time), so start on a few busy lanes, not nationally. Risks: disintermediation after the first load, trust and safety, and slow payment.

Alternative solution. Add a third side, drivers, if they are distinct from carriers.

Tradeoffs. Starting on busy lanes is slow to expand and quick to prove.

EX-C19-03 · How many suppliers for 90%?

Problem. Each supplier fulfills a request with probability 2%. How many suppliers does a market need for an 85% match rate? If each supplier's chance is 8%, how many for 95%?

Context. Cold-start planning.

Data. $P(\text{match}) = 1 - (1 - p)^N$.

Constraints. Five minutes.

Expected approach. FW-C19-03.

Solution. For $p = 0.02$ and 85%: $N = \ln(0.15) \div \ln(0.98) = 93.9$, so 94 suppliers. For $p = 0.08$ and 95%: $N = \ln(0.05) \div \ln(0.92) = 35.9$, so 36. A smaller per-supplier chance requires far more suppliers: a quarter of the chance needs about 2.6 times the suppliers for a lower target. The lesson: raising each supplier's chance of fulfilling (by narrowing the offering, improving matching or availability) can reduce the density needed more than recruiting would.

Alternative solution. Narrow the category so p rises from 2% to 5%: the need drops to $\ln(0.15) \div \ln(0.95) = 37$ suppliers.

Tradeoffs. Narrowing reduces the addressable market and makes the first market work.

EX-C19-04 · The developer funnel

Problem. 8,000 developers sign up. 75% get credentials, 65% of those make a first successful call, 60% of those reach production. Compute the end-to-end share and count. If the steps are improved to 80%, 75% and 65%, what is the new share and count?

Context. A developer platform review.

Data. As above.

Constraints. Five minutes.

Expected approach. FW-C19-04.

Solution. $8,000 \times 0.75 = 6,000$; $\times 0.65 = 3,900$; $\times 0.60 = 2,340$: 29.25%. Improved: $8,000 \times 0.80 \times 0.75 \times 0.65 = 3,120$: 39%. The weakest step in relative terms is the first call (65%); raising it from 65% to 75% adds 10 points of a base of 6,000, or 600 developers at that stage, which flows to 360 in production (at 60%). Look at error messages and the first example.

Alternative solution. Improve the last step first (production), since a developer who is already integrated is closest to value.

Tradeoffs. The step with the biggest drop is not always the cheapest to fix; check both.

EX-C19-05 · A take-rate model

Problem. At a 25% take rate, transaction value is 100; at 18%, 110; at 12%, 122. Operating cost is 7% of transaction value. Compute revenue and contribution at each. What would justify the 12% rate?

Context. A pricing review.

Data. As above.

Constraints. Five minutes.

Expected approach. Worked example 3.

Solution. 25%: revenue 25.0, cost 7.0, contribution 18.0. 18%: revenue 19.8, cost 7.7, contribution 12.1. 12%: revenue 14.64, cost 8.54, contribution 6.1. Contribution falls by two thirds from 25% to 12%. The lower rate is justified only if it buys something else: faster ecosystem growth that raises the value of the platform's other products, a defense against a rival with a lower rate, regulation, or developer goodwill that has measurable effects on retention. Quantify that benefit and compare it with the lost contribution of about 12 per 100.

Alternative solution. Tier the rate: 25% above a threshold of sales, 12% for small participants.

Tradeoffs. A cut cannot be reversed easily; raise rates rarely and with notice.

EX-C19-06 · Reviewer capacity

Problem. An app store receives 100,000 submissions a week. 70% are low-risk and automated; 25% are medium risk at 0.5 reviewer-hours each; 5% are high risk at 3 hours each. Reviewers work 35 productive hours a week. How many reviewers are needed? What if automation moves 10 points of submissions from medium to low?

Context. Staffing a review policy.

Data. As above.

Constraints. Five minutes.

Expected approach. IQ-C19-05.

Solution. Medium: $25,000 \times 0.5 = 12,500$ hours. High: $5,000 \times 3 = 15,000$ hours. Total 27,500 hours; $\div 35 = 786$ reviewers. If 10 points of submissions (10,000) move from medium to low: medium $15,000 \times 0.5 = 7,500$ hours; total 22,500 hours; $\div 35 = 643$ reviewers, a saving of 143 reviewers. The high-risk tier is more than half the effort for 5% of submissions, which is where specialist skill and good triage pay most.

Alternative solution. Reduce the high-risk time with better tooling and checklists.

Tradeoffs. Moving submissions to the low tier saves cost and raises the risk that something harmful passes; monitor the harmful-approval rate.

EX-C19-07 · Concentration risk

Problem. The top 10 developers produce 40% of a platform's revenue, and the largest produces 12%. What are the risks, what indicators would you watch, and what would you do?

Context. An ecosystem risk review.

Data. As above.

Constraints. Six minutes.

Expected approach. FW-C19-06.

Solution. Risks: the platform depends on a few developers, who have leverage over rates and roadmap, and whose departure (or a competing offer) would cut revenue and may pull users away. If the largest leaves, 12% of revenue is at risk. Indicators: the share of revenue from the top 1 and top 10 over time; the top developers' use of rival platforms (multi-homing); their growth versus the rest; contract and renewal dates; sentiment in meetings. Actions: grow the number of mid-sized developers (support, programs), offer volume tiers open to all, keep a regular, structured relationship with top developers without special deals that hurt trust, and track whether the top-10 share falls. Set an alert if the top-10 share rises above a threshold.

Alternative solution. Diversify with a program to help new developers reach a first meaningful revenue level.

Tradeoffs. Heavy investment in the top accounts increases dependence; balance with the broader base.

EX-C19-08 · Disintermediation arithmetic

Problem. A marketplace has 50,000 first jobs a month; 35% of pairs would rebook (17,500); 40% of those leak off-platform (7,000). The take rate is 12% on a \$50 job (\$6). Compute monthly revenue lost to leakage. Test two interventions: (a) halve the rate on repeat jobs to 6%, cutting leakage to 15%; (b) add a guarantee costing \$1 per job that cuts leakage to 20%.

Context. A marketplace fighting leakage.

Data. As above.

Constraints. Seven minutes.

Expected approach. IQ-C19-06.

Solution. Today: on-platform repeat jobs $17,500 - 7,000 = 10,500 \times \$6 = \$63,000$; leakage loses $7,000 \times \$6 = \$42,000$. (a) Leakage 15% leaves $17,500 \times 0.85 = 14,875$ on-platform $\times \$3 = \$44,625$: less than \$63,000; the discount does not pay. To match \$63,000 at \$3 would need 21,000 jobs, more than the 17,500 that exist. (b) Leakage 20% leaves 14,000 on-platform $\times (\$6 - \$1) = \$70,000$: more than \$63,000. So the guarantee pays and the discount does not. The principle: reduce leakage by adding continuing value, not by cutting the price of the introduction.

Alternative solution. Combine a small guarantee with recurring-booking tools.

Tradeoffs. A guarantee has a cost and a risk of claims; price it from claim rates.

EX-C19-09 · Platform metrics for a ride-hailing service

Problem. List a primary metric and metrics for supply, demand, interaction, quality, economics and ecosystem health for a ride-hailing platform, and two early warnings.

Context. A health dashboard.

Data. None.

Constraints. Seven minutes.

Expected approach. FW-C19-07.

Solution. *Primary:* completed trips per ride request. *Supply:* active drivers, hours online, new drivers per week. *Demand:* active riders, requests, new riders. *Interaction:* request fill rate, time to pickup, cancellation rate. *Quality:* ratings, safety incidents per 100,000 trips, disputes.

Economics: gross bookings, take rate, contribution per trip, driver earnings per hour.

Ecosystem health: share of trips by the top drivers; driver churn by cohort; multi-homing (drivers on rival apps). *Early warnings:* rising pickup times with flat driver counts (supply thinning where demand is); falling driver earnings per hour (a leading indicator of driver churn). Guardrails: safety and driver earnings must not be sacrificed for rider growth.

Alternative solution. Add a measure of surge frequency as an indicator of imbalance.

Tradeoffs. Many metrics invite confusion; keep one primary and a few guardrails.

EX-C19-10 · A timed full answer

Problem. Give a 25-minute answer to: "A car maker wants to open its in-car screen to third-party apps. Design the platform: sides, cold start, review, revenue and risks."

Context. Self-scored; record yourself. State assumptions aloud.

Data. None. Label what you assume.

Constraints. Use the 25-minute budget in section 4.

Expected approach. The platform answer structure, scored against the six behaviors in section 1.

Solution. An outline of a strong answer. Sides: drivers (want useful, safe apps), developers (want reach, tools and revenue), the car maker (brand, safety, revenue), and regulators (driver distraction). Special constraint: safety in a moving vehicle: apps must limit distraction, so the review is stricter than on a phone, and certain app types (video while driving) are blocked. Cold start: a small set of launch partners in the highest-value categories (navigation, music, charging, parking), a developer kit with a simulator, and single-player value from the car maker's own apps; developer incentives. Review: tiered, with distraction testing and a driving-safety guideline; fast updates for low-risk apps.

Revenue: a lower take rate initially to attract developers, rising with the platform's reach; subscriptions bundled with the car; data rules (privacy). Risks: safety incidents; fragmentation across car models; the phone's dominance (drivers project their phones), which the car maker must either embrace or compete with; regulatory attention; long vehicle lifecycles (software must be updatable). Metrics: production-active developers, trips with apps used, distraction incidents, developer time to first app. Sequence: pilot with a few models and partners.

Alternative solution. Choose to support phone projection first and add native apps later, because drivers already use their phones.

Tradeoffs. A closed approach protects safety and limits developers; an open one invites innovation and risk.

13. Practice protocol

Timed drills (three a week for three weeks).

- *Week 1: sides and the cold start.* Use IQ-C19-01, IQ-C19-02, EX-C19-01, EX-C19-02 and EX-C19-03. Draw the map, compute the density needed and say the narrow-launch plan aloud.
- *Week 2: developers and economics.* Use IQ-C19-03, IQ-C19-04, EX-C19-04, EX-C19-05 and EX-C19-08. Compute funnels, take-rate responses and leakage arithmetic by hand.
- *Week 3: governance, risk and metrics.* Use IQ-C19-05, IQ-C19-06, IQ-C19-07, EX-C19-06, EX-C19-07, EX-C19-09 and EX-C19-10. Design a review policy with capacity numbers, scan the risks, and run a full 25-minute answer with a partner who changes the stage midway.

With a partner. Ask your partner to say "It has network effects, so it'll grow" and "Why not just cut the fee?" at random. Practice naming the mechanism and its limits, and running the arithmetic.

Study a real platform. Read the public rules and fee schedule of one platform you use, map the sides, and list two governance tradeoffs you can see in the document.

Feedback checklist (yes, partly, no).

1. Did I map every side's gives, gets, alternatives and fears?
2. Did I classify the platform and name its hard problems?

3. Did I treat network effects as a mechanism with limits, and compute density for a cold start?
4. Did I treat the API as a product and measure the first mile?
5. Did I set governance as a three-way tradeoff and model the take rate with a cost floor?
6. Did I scan dependency, concentration, disintermediation, conflict and regulation?
7. Did I choose a primary metric tied to successful transactions or production use, with guardrails?
8. Did I say what would change my recommendation?
9. Did I avoid the label "network effects" without a mechanism?
10. Did I finish with the two-sentence close from section 4?

Answer journal. For each drill record the date, the platform, the sides, the main tradeoff, the key number, your score out of ten and the step you rushed.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. Each deliverable doubles as an interview story. If you do not work on a platform, apply it to your product's relationships with partners, integrations or an internal platform you use.

Days 1 to 3. Map your product's sides (FW-C19-01): who gives and gets what, their alternatives and fears. If the map is one-sided, note it and ask a partner or user about the other side.

Week 1. Find the first-mile funnel for anyone who builds on or integrates with your product (an API, a partner program, an internal tool). Measure or estimate the time to first success and the share who reach production or adoption.

Week 1. Compute the density or coverage your product needs in a market or segment for matching or fulfillment to work, using the formula, with the best available estimate of p .

Week 2. Write down the governance rules of your product's ecosystem (quality bar, fees, review) and the three-way tradeoff each makes. Ask a participant what they find unfair or opaque.

Week 2. Run the risk scan (FW-C19-06): concentration, dependency, leakage, conflict with participants. Pick one indicator to track.

Week 3. Define the platform health metrics for your area: a primary metric and guardrails across sides, and one early warning with an owner.

Deliverables that become stories: the platform map (a time you saw a problem from the other side), the first-mile funnel (a time you improved a developer or partner experience), and the density or leakage analysis (a time you used a simple model to decide where to focus). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. A platform serves more than one group whose value depends on each other; map each side's gives, gets, alternatives and fears, and classify the platform (two-sided market, developer platform or both) to know which problems it faces.
2. Network effects are mechanisms with limits (local, saturating, undermined by multi-homing); for a cold start, concentrate supply where density is reachable, using the matching probability, and measure liquidity, not sign-ups.
3. Treat an API as a product: measure the first mile (time to first successful call, share reaching production), protect stability with versioning and deprecation, and read developer experience as the adoption lever.
4. Governance is a three-way tradeoff among speed, safety and trust, plus openness and price; model the take rate against participation and the cost floor, tier review by risk, and explain and allow appeals.
5. Scan for ecosystem risk (dependency, concentration, disintermediation, conflict with participants, regulation), fight leakage with continuing value and not only price, and run the platform on a primary metric tied to successful transactions with guardrails across sides.

Further reading

- Geoffrey G. Parker, Marshall W. Van Alstyne and Sangeet Paul Choudary, *Platform Revolution* (2016). Platform design and governance [V: W. W. Norton, 28 March 2016, ISBN 9780393249132].
- David S. Evans and Richard Schmalensee, *Matchmakers: The New Economics of Multisided Platforms* (2016) [V: Harvard Business Review Press, 24 May 2016, ISBN 9781633691728].
- Jean-Charles Rochet and Jean Tirole, "Platform Competition in Two-Sided Markets," *Journal of the European Economic Association* (2003), the academic foundation for pricing on two-sided platforms [V: Journal of the European Economic Association 1(4), 990 to 1029, June 2003, doi 10.1162/154247603322493212].
- Andrew Chen, *The Cold Start Problem* (2021), on launching networks [V: Harper Business, 7 December 2021, ISBN 9780062969743].
- Apple, *App Review Guidelines* (developer.apple.com); Apple's press releases of 14 July 2008 and 18 November 2020 (apple.com/newsroom); Google, "Boosting developer success on Google Play," Android Developers Blog, 16 March 2021; and Apple's Form 10-Q for the quarter ended June 2026 (SEC). Primary sources for CS-C19-02 [as of 2026-10].
- The public documentation and changelogs of major API platforms, as worked examples of developer experience and versioning [as of 2026-10].

- Ron Kohavi, Diane Tang and Ya Xu, *Trustworthy Online Controlled Experiments* (2020), on interference when testing changes in two-sided markets [V: Cambridge University Press, 2 April 2020, ISBN 9781108724265; Twyman's law is chapter 3].